

MiniCode Python — 面试拷打手册

从面试官视角出发，覆盖项目架构、简历亮点、常见追问。
每道题给出**答题思路**，辅以关键代码片段，不堆代码。

一、项目整体理解

Q1：用一句话描述这个项目是什么，它解决了什么问题？

答：
MiniCode Python 是一个终端 AI 编程助手，核心问题是：如何让 LLM 在终端中自主完成多步编程任务——读文件、改代码、跑命令——同时保证安全可控。
类比：Claude Code 的开源 Python 实现，去掉商业层，保留核心 Agent 能力。

Q2：整体架构是怎样的？各模块怎么分工？



数据流向：用户输入 → TUI 捕获 → agent_loop 启动 → 模型返回 AgentStep → 工具执行 → 结果追加 messages → 下一轮循环。

Q3：为什么用 Python 而不是 TypeScript（原版是 TS 的）？

答：
Python 生态在 AI 工具链上天然友好：`anthropic / openai` SDK、`ast` 模块做代码分析、`subprocess` 做进程管理。更重要的是，Python 版作为开源实现，便于社区二次开发和学习。TypeScript 原版是商业产品，Python 版是学习和实验的载体。

二、Agent 执行引擎（简历重点一）

简历描述：实现多步工具调用闭环，支持只读工具并发执行与串行写操作分离，具备空响应自动重试、上下文窗口超限自动压缩等容错机制。

Q4: Agent 的核心循环是怎么工作的？完整流程讲一下。

答：

入口是 `run_agent_turn()`，它是整个助手的"心脏"，运行在一个独立的后台线程里。

完整执行流程：

```
用户输入
↓
TUI 把输入追加到 messages 列表 (role=user)
↓
run_agent_turn 启动 while 循环 (最多 max_steps=50 轮)
|
|─ Step N:
|   1. fire_hook_sync(AGENT_START)           ← 通知 Hooks
|   2. model.next(messages)                  ← 调 LLM, 返回 AgentStep
|   3. 判断 AgentStep 类型:
|       |─ type="assistant" (纯文本) → 追加 messages, return 结束循环
|       |─ type="tool_calls" (工具调用) → 执行工具
|           |─ 分类: concurrent_calls / serial_calls
|           |─ 并发执行 concurrent_calls (ThreadPoolExecutor)
|           |─ 串行执行 serial_calls (按顺序)
|           |─ 按原始 call id 排序, 合并结果
|           |─ 追加 assistant_tool_call + tool_result 到 messages
|   4. fire_hook_sync(AGENT_STOP/POST_TOOL_USE)
|   5. continue → 进入下一轮 (带上完整 messages 历史)
|
|─ 超出 max_steps → 返回 fallback 消息
```

关键设计：messages 是唯一的共享状态。 模型每轮都读完整对话历史（含所有工具调用结果）来决策下一步，不需要额外的"任务状态机"。这是 ReAct 范式（Reasoning + Acting）的 Python 实现。

与 TUI 的交互： `run_agent_turn` 通过三个 callback 和 UI 解耦：

- `on_tool_start`：工具开始时刷新 UI 状态
- `on_tool_result`：工具完成后把结果推到 transcript
- `on_assistant_message`：模型回复后渲染到屏幕

这样 Agent 线程和渲染线程完全解耦，互不阻塞。

Q5: 你说"只读工具并发、写操作串行"，具体怎么实现的？

答：

工具元数据里有 `is_concurrency_safe` 标志（通过 `ToolCapability.CONCURRENCY_SAFE` 设置）。`agent_loop` 在执行多工具调用时先做分类：

```
for call in calls:
    tool_def = tools.find(call["toolName"])
    if tool_def and tool_def.is_concurrency_safe:
        concurrent_calls.append(call)
    else:
        serial_calls.append(call)
```

并发组用 `ThreadPoolExecutor` 同时跑，最多 8 个线程；串行组按原始顺序逐个执行。最后按原始调用顺序合并结果，保证 messages 的顺序不乱。

Q6：空响应重试和上下文压缩是怎么做的？

答（分两块）：

空响应重试：模型偶尔返回空字符串（网络抖动、thinking 被截断等）。代码里有三个重试场景：

```
# 场景1：普通空响应，最多重试2次
if is_empty and empty_response_retry_count < 2:
    messages.append({"role": "user", "content": NUDGE_AFTER_EMPTY_RESPONSE})
    continue

# 场景2：thinking 阶段被 max_tokens 截断，最多重试3次
if _is_recoverable_thinking_stop(...) and recoverable_thinking_retry_count < 3:
    messages.append({"role": "user", "content": RESUME_AFTER_MAX_TOKENS})
    continue
```

重试的本质是注入一条“催促”消息，让模型继续，而不是重新发请求。

上下文压缩：`ContextManager` 估算当前 token 数（字符数 ÷ 4），超过 95% 阈值时触发 `compact_messages()`，策略是保留 system prompt + 最近 N 条消息，删去中间的历史。

Q7：工具执行失败了怎么处理？

答：

两层保护：

1. **ToolRegistry 层：**每个工具 `execute()` 有 try/except，失败返回 `ToolResult(ok=False, output=错误信息)` 而不是抛异常
2. **agent_loop 层：**`_execute_single_tool` 有全局安全网，捕获任何未预期异常，转成错误 `ToolResult`，同时重置 `store` 状态（防止 UI 卡在 "busy"）

（store 是全局状态容器，类型是 `Store[AppState]`，存着 TUI 需要实时展示的运行状态，比如当前是否 busy、正在执行哪个工具、已用多少 token。

TUI 主线程靠读这个 store 来决定渲染什么。如果工具崩了没有重置 store，`is_busy=True` 就永远留在那，UI 会一直显示“正在执行 xxx”，用户看到的界面就卡死了。）

工具错误不会终止循环——错误信息会作为 `tool_result` 传给模型，模型自己决定是重试还是换策略。

三、工具生态（简历重点二）

简历描述：内置 40+ 工具，通过 ToolRegistry 统一注册，支持动态加载 MCP 工具和本地 Skill 扩展。

Q9：工具输出太大怎么办？

答：

`_smart_truncate_output` 针对不同工具类型有不同截断策略：

- `read_file`：保留头 60% + 尾 40%，中间替换为 `[N lines omitted]`
- `run_command`：保留头尾 + 从中间提取包含 `error/fail/exception` 的关键行
- `grep_files`：保留前 N 条匹配 + 总行数提示

这样截断后模型仍能得到最关键的信息（文件开头的 import/声明、命令的报错信息），而不是被硬切在随机位置。

Q10：MCP 是什么，怎么集成进来的？

答：

MCP (Model Context Protocol) 是 Anthropic 提出的开放协议，定义了 LLM 宿主和外部工具服务之间的通信标准，类似 LSP (Language Server Protocol) 之于编辑器。

集成架构：

```
settings.json 配置
{"mcpServers": {"github": {"command": "npx", "args": ["@github/mcp"]}}}
↓
启动时 create_mcp_backed_tools() 读取配置
↓
为每个 server 创建 StdioMcpClient（懒加载，首次调用才启动进程）
↓
子进程通过 stdin/stdout 用 JSON-RPC 2.0 通信
↓
发送 initialize → 协商协议版本
发送 tools/list → 获取工具列表
↓
把每个 MCP 工具转成本地 ToolDefinition 注册进 ToolRegistry
```

懒加载设计：`StdioMcpClient` 配置了但不用时不启动子进程，第一次调用 `_ensure_started()` 才真正 `subprocess.Popen`，减少启动开销。

协议协商：支持两种传输格式——`content-length` (HTTP-like 头部帧) 和 `newline-json` (换行分隔 JSON)，启动时自动尝试，兼容不同 MCP 服务端实现。

安全措施：MCP 命令有白名单（只允许 `node/python/npx` 等），参数禁止包含 `|&$( )` 等 shell 元字符，防止命令注入。

对 `agent_loop` 完全透明——它看到的 MCP 工具和内置工具没有任何区别。

Q10b: Skills 是怎么工作的？和 MCP 有什么区别？

答：

Skills 是更轻量的扩展机制——本质是一个放在特定目录下的 `SKILL.md` Markdown 文件，描述一项可复用的任务模板或操作规程。

发现机制：

```
# 按优先级搜索四个目录（project 优先于 user）
skill_roots = [
    .mini-code/skills/      # 项目级（project 优先）
    ~/.mini-code/skills/    # 用户级
    .claude/skills/         # 兼容 Claude Code 项目格式
    ~/.claude/skills/       # 兼容 Claude Code 用户格式
]
```

`discover_skills(cwd)` 扫描这些目录，找到所有 `<名称>/SKILL.md`，提取第一段非标题文字作为 description，注册到 ToolRegistry 里。

使用流程：AI 调用 `load_skill("skill名称")` 工具，`SKILL.md` 的完整内容作为工具结果返回给模型，模型据此执行技能描述的步骤。

与 MCP 的区别：

维度	Skills	MCP
形式	Markdown 文件	独立进程 (JSON-RPC)
内容	任务模板 / 操作规程	实际工具调用能力
扩展成本	写 Markdown 即可	需要实现 MCP 服务器
典型场景	"如何部署这个项目"	"查询 GitHub Issues"

Skills 是"告诉模型怎么做"，MCP 是"给模型新的做事能力"。

四、TUI 与交互安全（简历重点三）

简历描述：事件驱动全屏终端界面，文件修改前展示 unified diff 供审批，权限系统支持按路径/命令粒度控制，审批支持快捷键和拒绝反馈给模型。

Q11: 全屏 TUI 是怎么实现的？用了什么框架？

答：

没有用 `rich/textual` 等框架，是手写的原始终端控制——直接写 ANSI 转义码操控光标和颜色。

架构是事件驱动：

主线程：读 `stdin` 字节流 → `parse_input_chunk()` 解析为 `KeyEvent/TextEvent`

↓

`_handle_event()` 路由处理

↓

`state` 变更 → `_render_screen()` 重绘

Agent 线程：`run_agent_turn()` 执行，回调更新 `state` → 触发重绘

`_ThrottledRenderer` 对重绘做限流（最小间隔 16ms，约 60fps），避免 Agent 高频回调时界面闪烁。

Q12：权限系统是怎么设计的？怎么防止 AI 越权操作？

答：

三层防线：

第一层：路径白名单。 `PermissionManager` 记录允许的目录，每次文件操作前检查目标路径是否在 `cwd` 内（用 `Path.resolve()` 防止 `../..` 穿越）。

第二层：操作分级。 `auto_mode.py` 把操作按风险分为 SAFE/LOW/MEDIUM/HIGH/DANGEROUS 五级：

```
SAFE_TOOLS = {"read_file", "list_files", "grep_files"} # 自动通过
# run_command 运行 rm -rf 之类命令 → HIGH → 必须用户确认
```

第三层：用户审批 + 反馈。 危险操作弹审批窗口，用户可以：

- 允许一次 / 本轮允许全部
- 拒绝并给模型反馈——用户写的拒绝理由会作为 `user` 消息注入对话，模型据此调整策略

最后一个是关键：不只是阻止，而是把人的意图传回给模型。

Q13：diff 预览是怎么生成的？

答：

用 Python 标准库 `difflib.unified_diff`，对比文件修改前后内容，生成标准 unified diff 格式（和 `git diff` 输出一致）。TUI 渲染时对 `+` 行染绿色、`-` 行染红色。

用户在审批窗口可以 `Ctrl+O` 展开/收起 diff，支持滚动，避免大文件 diff 把屏幕撑满。

五、Hooks 与可扩展性（简历重点四）

简历描述：参照 Claude Code 设计实现完整生命周期 Hook 系统，支持三层持久化记忆（TF-IDF 检索）注入系统 Prompt。

Q14: Hooks 系统是怎么设计的？有什么用？

答：

hooks.py 定义了生命周期事件枚举：

```
class HookEvent(str, Enum):
    PRE_TOOL_USE = "pre_tool_use" # 工具调用前
    POST_TOOL_USE = "post_tool_use" # 工具调用后
    AGENT_START = "agent_start" # Agent 轮开始
    AGENT_STOP = "agent_stop" # Agent 轮结束（含异常）
    SESSION_SAVE = "session_save" # 会话保存
```

用法：外部脚本或插件通过 register_hook(event, handler) 注册回调，在对应事件触发时执行。
agent_loop 里 fire_hook_sync(AGENT_STOP, ...) 在 finally 块里调用，保证即使 Agent 崩溃也能触发（比如记录日志、发通知）。

实际价值：审计工具调用、自动记录日志、接入外部监控，不需要改 Agent 核心代码。

Q15: 三层记忆是怎么工作的？为什么用 TF-IDF？

答：

三层结构：

层级	存储位置	作用范围
User	~/mini-code/memory/	跨所有项目
Project	.mini-code-memory/	当前项目，可提交 git
Local	.mini-code-memory-local/	当前项目，不提交

每条记忆是一个 JSON 文件，包含内容、标签、创建时间。启动时记忆被读出，相关的注入 system prompt。

为什么 TF-IDF：记忆条目可能几十上百条，不能全量注入（浪费 token）。TF-IDF 按用户当前问题与记忆内容的词频相关性打分，选出最相关的 Top-K 条注入。

```
# 核心打分：query 词的 TF × IDF 的点积
score = sum(tf_doc[term] * idf[term] for term in query_tokens if term in tf_doc)
```

不用 embedding 向量检索的原因：纯本地运行，不依赖外部 API，TF-IDF 对几百条短文本已经足够准确。

六、综合追问

Q16：这个项目和 Claude Code 的核心差异是什么？

答（诚实作答）：

维度	Claude Code	MiniCode Python
代码质量	生产级，闭源	开源，实验性
工具完整度	更完整（glob、notebook 等）	覆盖主要工具
流式输出	完善	基本支持
多模型	仅 Claude	Anthropic / OpenAI / OpenRouter
学习价值	不可读	可读，可改

MiniCode Python 的价值不在于对标 Claude Code，而在于**可读、可改、可学习的 Agent 架构参考实现**。

Q17：项目里最难解决的技术问题是什么？

推荐答法（选其一）：

并发工具执行的结果顺序问题：多个工具并发执行完成顺序不确定，但 LLM 对话历史里工具调用和结果必须——对应且顺序一致。解法是用 `call["id"]` 建索引，结果收集后按原始调用顺序排序再追加 messages。

TUI 输入事件死锁：审批弹窗弹出时，输入事件处理和 Agent 线程共享状态，早期版本会出现按键无响应。解法是把输入事件串行化处理（单线程队列），Agent 线程只能通过 callback 更新 state，不直接读写输入状态。

Q18：如果让你重构这个项目，你会改什么？

推荐思路（体现反思能力）：

- 类型系统：**`ChatMessage` 是 `TypedDict`，但 `role` 有 6 种，不同 `role` 的字段完全不同，用 `Union` 类型或 `dataclass` 更安全
- 工具输出截断：**目前按字符数估算 token，可以接入模型的 `tokenizer` 做精确计数
- 并发模型：**当前用 `ThreadPoolExecutor`，对 IO 密集工具（`web_fetch`）合适，未来可换 `asyncio` 统一并发模型

附：快速记忆卡片

问题关键词	核心答案
Agent 循环	<code>while + model.next + tools.execute + messages</code> 追加
并发工具	<code>is_concurrency_safe</code> 分类 + <code>ThreadPoolExecutor</code> + 按 <code>id</code> 排序合并

问题关键词	核心答案
空响应重试	注入 NUDGE 消息，最多 2-3 次，不重发请求
上下文压缩	估算 95% 阈值，compact = system + 最近 N 条
权限系统	路径白名单 + 风险分级 + 用户审批 + 拒绝反馈给模型
TUI 架构	主线程读输入 + Agent 子线程 + ThrottledRenderer 限流重绘
记忆检索	TF-IDF 打分，本地文件存储，三层作用域
MCP	stdio 启动子进程，工具动态注册进 ToolRegistry